

CareLite Healthcare Learning Platform

High Level Design Document

Architecture overview for EKS, Cognito, S3, schema-based tenancy, Jenkins, Argo CD, and observability.

Field	Value
Document Type	HLD
Version	0.1
Date	2026-05-15
Status	Draft for learning and implementation planning
Primary Stack	Java 21, Spring Boot 3.3.x, Spring Cloud 2023.0.x, Maven 15-module monorepo, Spring Cloud Gateway, OpenFeign, Amazon MSK/Kafka, MongoDB Atlas, AWS EKS, Cognito, S3, WAF, Shield, Jenkins, Argo CD

Document Control

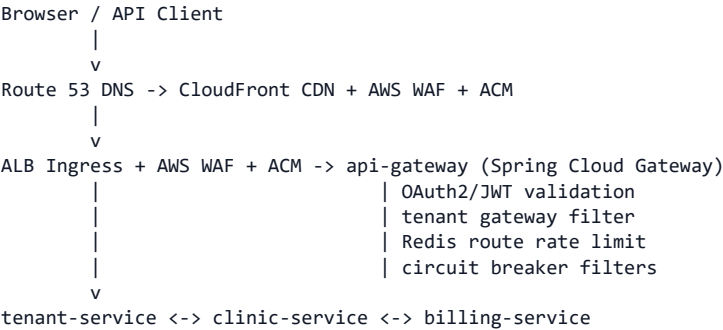
Item	Details
Owner	CareLite learning project team
Audience	Developer, DevOps engineer, solution architect, product owner
Purpose	Guide a small healthcare platform that teaches production microservice patterns.
Scope Level	MVP plus platform learning patterns

1. Architecture Summary

CareLite uses a small microservice architecture deployed on AWS EKS. The architecture favors real production patterns while keeping the business domain narrow. Runtime services communicate synchronously through REST for user workflows and asynchronously through Kafka for domain events.

1.1 Verified Platform Stack

Layer	Choice
Language and framework	Java 21 with virtual threads and records; Spring Boot 3.3.x; Spring Cloud 2023.0.x; Maven reactor with 15 modules.
Gateway and REST	Spring Cloud Gateway, Spring Security OAuth2 resource server, OpenFeign, Resilience4j circuit breakers.
Data	RDS PostgreSQL Multi-AZ per-tenant schemas, MongoDB Atlas event store and CQRS read models, ElastiCache Redis, S3.
Messaging	Amazon MSK/Kafka with 48 partitions for domain.events, Apache Avro, Confluent Schema Registry, Transactional Outbox.
Security	AWS Cognito OAuth2, custom JWT claims, IAM/IRSA, WAF, Shield, Falco runtime monitoring, Trivy CVE scanning.
Architecture patterns	Event Sourcing, CQRS, Saga orchestration, Outbox, Idempotency, per-schema multi-tenancy, DDD bounded contexts.
Observability	Prometheus, Grafana, Loki, Fluent Bit, OpenTelemetry, Tempo, Alertmanager, Grafana Faro.
Batch and documents	Spring Batch with 5 jobs, iText 8 PDFs, Apache POI Excel, EventArchivalJob to Amazon S3 Avro archive.
CI/CD and quality	Jenkins, Argo CD GitOps, Google Jib, SonarQube, Flyway, Testcontainers, ArchUnit, WireMock.



**Data plane:**

RDS PostgreSQL: master schema + one schema per clinic tenant

S3: tenant-scoped document prefixes

Kafka: domain events and outbox relay

MongoDB: audit/event store and replay records

MongoDB: CQRS read models for audit, appointment timeline, and billing views

Redis: tenant lookup cache and short-lived locks

Delivery plane:

Jenkins -> ECR -> Git manifests -> Argo CD -> EKS

2. Deployment View

Component	AWS or OSS Service	Purpose
Runtime	Amazon EKS	Runs Spring Boot workloads and observability agents.
DNS	Amazon Route 53	Tenant DNS, health checks, weighted/failover records. Not used for HTTP rate limiting.
CDN and edge	Amazon CloudFront + AWS WAF + ACM	TLS termination, caching, bot/rate-based edge controls.
Ingress	AWS Load Balancer Controller + ALB + ACM	Routes external HTTPS traffic to api-gateway with WAF protection.
Images	Amazon ECR	Stores service container images.
Database	Amazon RDS PostgreSQL	Master schema and tenant schemas.
Event Store	MongoDB Atlas on AWS	Immutable audit/event stream, CQRS read models, replay records.
Cache and locks	Amazon ElastiCache Redis	Tenant cache, distributed locks, and gateway rate limiter state.
Messaging	Amazon MSK/Kafka	domain.events topic with 48 partitions and tenant+aggregate partition keys.
Schema Registry	Confluent Schema Registry	Avro schema versioning and compatibility checks.
Object Storage	Amazon S3	Tenant documents and exports.
Identity	Amazon Cognito	OAuth2 login, users, groups, JWT tokens.
Secrets	AWS Secrets Manager	Database and Cognito secrets.
Runtime security	Falco	Detects suspicious container and node behavior.
CI	Jenkins	Build, test, scan, package, push image.
CD	Argo CD	GitOps deployment to EKS.

3. Service Responsibilities

Service	Main Responsibilities	Storage
api-gateway	Spring Cloud Gateway routes, validate Cognito JWT, resolve tenant, add headers, Redis rate limits, circuit breaker filters.	Redis cache and limiter state.
tenant-service	Tenant registry, schema provisioning, Cognito setup, S3 prefix setup.	RDS master schema.
clinic-service	Patients, doctors, appointments, notes, document metadata; exposes Feign-safe internal APIs.	RDS tenant schema, S3.
billing-service	Invoices, payment status, appointment billing saga, Feign client to clinic-service with circuit breaker.	RDS tenant schema.
audit-event-service	Consume Kafka events, audit queries, replay learning flow.	MongoDB event store.

4. Tenant Routing

- Clinic clients use subdomains such as apollo.carelite.example.
- api-gateway extracts slug apollo from Host header.
- Gateway checks Redis cache, then tenant-service, to map slug to tenant_id clinic_apollo.
- Gateway forwards X-Tenant-ID to downstream services.
- Gateway also propagates X-Correlation-ID, authenticated actor, role hints, and trace context.
- Each service sets PostgreSQL search_path to the tenant schema for the request transaction.
- Services must reject requests when X-Tenant-ID is missing or invalid.

5. Rate Limiting and Resilience

Layer	Mechanism
Route 53	DNS health checks, failover, weighted routing. It does not enforce per-request rate limits.
CloudFront	CDN caching, origin shielding, ACM TLS, AWS WAF rate-based and managed rules.
ALB Ingress	Kubernetes ingress through AWS Load Balancer Controller, ACM TLS, optional WAF association.
Spring Cloud Gateway	RedisRateLimiter per route and tenant, request-size limits, circuit breaker filter, retry filter for safe methods.
Feign clients	OpenFeign plus Resilience4j timeout, retry, circuit breaker, bulkhead, and fallback.
Kafka consumers	Consumer groups scale horizontally; KEDA can scale based on lag.

6. Tenant Onboarding Saga

1. Receive POST /tenants/onboard with Idempotency-Key.
2. Insert master.tenants row with status PROVISIONING.
3. Create tenant schema: clinic_<slug>.
4. Run Flyway migrations for the new schema.
5. Create S3 folder marker and tenant prefix policy convention.
6. Create Cognito groups and first admin user.
7. Seed clinic settings.
8. Mark tenant ACTIVE.
9. Publish TENANT_ONBOARDED through outbox.

7. Security Architecture

- OAuth2 Authorization Code with PKCE through Cognito Hosted UI.
- JWT validation at the gateway and optional resource-server validation in services.
- RBAC based on Cognito groups and custom claims.
- Tenant isolation enforced by schema search_path and application tenant checks.
- S3 object keys include tenants/{tenant_id}/ prefix.
- ACM certificates terminate HTTPS at CloudFront and ALB; internal service traffic can later add mTLS through a service mesh.
- Secrets stored in AWS Secrets Manager and mounted through External Secrets Operator or CSI driver.

8. Event Architecture

Pattern	Implementation
Outbox	Domain service writes business row and outbox_events row in same DB transaction.
Relay	Scheduled relay or Debezium-style worker publishes Avro events to Kafka.
Audit	audit-event-service consumes domain.events and writes append-only event documents to MongoDB.
CQRS	MongoDB read models are updated from Kafka events for audit and dashboard queries.
Replay	Replay marks event metadata.replay=true and suppresses unsafe side effects.
Idempotency	Consumers record applied_events before applying event side effects.
Retention	Hot events stay in MongoDB Atlas; EventArchivalJob writes nightly Avro archives to S3.

9. High Availability and Scalability

Concern	Design
EKS	Managed node groups across at least two Availability Zones, cluster autoscaler or Karpenter.
Services	At least two pod replicas, readiness/liveness probes, rolling updates, topology spread constraints.
Autoscaling	HPA on CPU/memory/request metrics; KEDA for Kafka lag on event consumers.
Database	RDS PostgreSQL Multi-AZ; connection pooling through Hikari and optional PgBouncer.
Event store	MongoDB replica set or MongoDB Atlas multi-AZ deployment with tenant/time indexes.
MSK	48 partitions on domain.events allow high parallelism while preserving tenant+aggregate ordering.
Secrets	Secrets Manager with least-privilege IAM roles for service accounts through IRSA.

10. CI/CD Architecture

```

Developer push
-> Jenkins pipeline
    -> Maven test/package
    -> Testcontainers integration tests
  
```

- > ArchUnit architecture tests
- > WireMock contract/resilience tests
- > SonarQube quality gate
- > Trivy filesystem and image scan
- > Google Jib image build
- > Push image to ECR
- > Update Helm values image tag in Git
- > Argo CD detects Git change
- > Syncs Helm chart to EKS
- > Kubernetes rollout

11. Observability

Concern	Tool	Notes
Metrics	Prometheus	Scrapes actuator /actuator/prometheus.
Dashboards	Grafana	Service health, JVM, API latency, tenant onboarding.
Logs	Loki plus Fluent Bit	Collects pod logs with trace ID and tenant ID fields.
Traces	Tempo plus OpenTelemetry	Distributed tracing across gateway and services.
Alerts	Alertmanager	High error rate, pod restart loop, onboarding failures.
Security scan	Trivy	Jenkins filesystem and image vulnerability scanning.

12. Java Design Patterns in Architecture

Pattern	Architecture Use
Chain of Responsibility	Spring Cloud Gateway filters for correlation, tenant, auth, rate limiting, and routing.
Adapter	Cognito, S3, MongoDB, Kafka, and payment provider integration boundaries.
Strategy	Tenant provisioning steps, rate limit policy selection, and document storage policy.
Circuit Breaker	Resilience4j wrappers around Feign clients and gateway routes.
Saga	Tenant onboarding and appointment-to-invoice workflow.
Observer	Outbox and Kafka event subscribers.

13. Key Tradeoffs

- Schema-per-tenant is excellent for learning isolation and provisioning, but migrations require discipline.
- Cognito is realistic for OAuth2 but local development needs mock JWT support or a dev user pool.
- Route 53 is kept for DNS and health routing; WAF and Spring Cloud Gateway own rate limiting.
- MongoDB makes event documents easy to model and query, but requires strong index and retention discipline.
- Jenkins plus Argo CD teaches a common enterprise split, but adds more setup than GitHub Actions alone.
- Five services keep learning manageable while still covering gateway, domain, billing, audit, and tenant provisioning.